

PROYECTO DE FIN DE CURSO · 2º DAW

Galería digital del IES Bernaldo de Quirós

Aplicación web full stack para administrar y exponer el catálogo de obras del instituto. Spring Boot · Angular · MySQL.

GALERÍA

AUTOR

Pelayo Prieto Calderón

DNI: 71781551T

CURSO ACADÉMICO

2025 / 2026

FECHA

12 de junio de 2026

Contenido

1.1. Palabras clave	4
Objetivo General	6
Objetivos Específicos.....	6
4.1. Análisis del contexto.....	7
4.2. Estado del arte.....	7
5..... Planificación	8
5.1. Recursos humanos y materiales	8
Recursos Humanos.....	8
Materiales	9
5.2. Presupuesto	10
6. Análisis del sistema	11
6.1. Definición de requisitos	11
Requisitos Funcionales.....	11
<i>Subsistema de gestión de obras (panel de administración)</i>	11
<i>Subsistema de gestión de autores (panel de administración)</i>	12
<i>Subsistema de gestión de técnicas (panel de administración)</i>	13
<i>Subsistema de consulta pública (visitante)</i>	13
Requisitos no funcionales	14
6.2. Identificación de los subsistemas.....	16
Subsistema de autenticación y autorización.....	16
Subsistema de gestión del catálogo.....	16
Subsistema de gestión de imágenes	17
Subsistema de consulta pública.....	18
Subsistema de autenticación y autorización.....	19

Subsistema de gestión del catálogo.....	20
Subsistema de gestión de imágenes	20
Subsistema de consulta pública.....	21
6.3. Análisis de casos de uso.....	22
6.4. Análisis de interfaces	23
Parte Administrador	23
Parte Pública.....	24
6.5. Especificación del plan de pruebas	24
Pruebas unitarias del backend	25
Pruebas de integración de la API REST	25
Pruebas funcionales end-to-end manuales	25
Pruebas de accesibilidad	26
7. Diseño del sistema	26
7.1. Arquitectura del sistema	26
Diagrama de despliegue	26
Diagrama de componentes del backend.....	27
Diagrama de componentes del frontend.....	29
7.2. Diseño de clases.....	31
7.3. Diseño de la base de datos	32
7.4. Diseño de interfaces.....	32
8.1. Lenguajes de programación	35
8.2. Herramientas utilizadas.....	36
9.1. Pruebas de código	36
TecnicaServiceTest (8 pruebas)	36
AutorServiceTest (8 pruebas)	37
ObraServiceTest (12 pruebas).....	37

9.2. Pruebas de accesibilidad	37
10.1. Desarrollo del proyecto.....	38
Implementación de la seguridad con Spring Security	38
Curva de aprendizaje de Angular	38
10.2. Futuras ampliaciones	39
Alta de autores, técnicas y materiales desde el formulario de obra	39
Filtrado avanzado del catálogo.....	39
Servicio de Email para la recuperación de contraseñas	40

1. Resumen

El I.E.S Bernaldo de Quirós cuenta con una colección de más de doscientas cincuenta obras de arte adquiridas a lo largo de los años, actualmente accesible al público únicamente mediante un PDF descargable de 42 páginas. Este formato presenta limitaciones de accesibilidad, interactividad y mantenimiento, además de romper la estética del resto de la web del instituto.

El presente proyecto consiste en el diseño e implementación de una aplicación web completa para la gestión y exposición digital de dicha colección. La solución sigue una arquitectura cliente-servidor con tres piezas claramente separadas: un cliente Angular 21 con diseño responsive, un backend Spring Boot 4 que expone una API REST, y una base de datos MySQL para la persistencia. Incorpora un panel administrativo protegido mediante autenticación JWT que permite gestionar el catálogo (obras, autores y técnicas) y subir las imágenes asociadas, así como una galería pública accesible donde cualquier visitante puede consultar la colección y filtrar por autor, año o título. La aplicación emplea exclusivamente software libre y de código abierto.

1.1. Palabras clave

Galería digital, museo virtual, catálogo de arte, aplicación web, Angular, Spring Boot, MySQL, API REST, JWT, autenticación, gestión de imágenes, accesibilidad web, cliente-servidor, IES Bernaldo de Quirós, patrimonio artístico.

2. Introducción

Actualmente el I.E.S Bernaldo de Quirós cuenta con un problema a la hora de hacer accesible su vasta galería de obras de forma digital. Cuentan con un pdf descargable en su página principal, el cual cuenta con 42 páginas con unas 8 obras cada una.

Uno de los mayores problemas con este enfoque sería el de la accesibilidad, el no poder apreciar las obras en un mayor tamaño/resolución y el como rompe el esquema del resto de la página web del instituto.

La solución más sencilla para arreglar todos estos problemas es el de realizar una página web interactiva, responsive y accesible que sea fácil de usar tanto para las personas interesadas en conocer la inmensa colección del instituto, como para los encargados de este de poder añadir, editar o eliminar obras sin la necesidad de rehacer un pdf nuevo por cada cambio.

3. Objetivos

Objetivo General

Como objetivo general me propuse el diseñar e implementar una aplicación web completa que permita la administración y exposición digital de las obras de la galería del I.E.S Bernaldo de Quirós, mediante una arquitectura cliente-servidor basada en Angular + Bootstrap para el frontend, Spring boot y MySQL para el backend.

Objetivos Específicos

1. Diseñar una interfaz web responsive adaptable a distintos dispositivos.
2. Implementar una arquitectura cliente-servidor separando frontend y backend.
3. Permitir el alta, modificación y eliminación de obras.
4. Incorporar la subida y almacenamiento de imágenes asociadas a las obras.
5. Implementar control de errores y manejo de excepciones.
6. Proporcionar una web para el usuario final que integre estándares de accesibilidad.
7. Establecer un sistema de autenticación para la administración de las obras mediante Spring Security.
8. Gestionar el catálogo museístico completo del instituto (obras, autores y técnicas) desde un panel de administración unificado.
9. Permitir la búsqueda y el filtrado de las obras del catálogo tanto en la zona pública como en el panel administrativo.
10. Garantizar la persistencia segura del catálogo mediante una base de datos relacional con borrado lógico, preservando el histórico de obras, autores y técnicas eliminadas.

4. Análisis del contexto y estado del arte

4.1. Análisis del contexto

El proyecto se enmarca en el I.E.S Bernaldo de Quirós, él cual cuenta con unas 250 obras (pinturas, grabados, fotografías, esculturas en metal y otras técnicas mixtas).

Actualmente, dicha colección se difunde mediante un documento PDF descargable de 42 páginas alojado en la página principal del instituto, donde cada página recoge alrededor de ocho obras con su ficha técnica básica y una imagen reducida. Aunque este enfoque permitió en su momento publicar el catálogo de forma sencilla, presenta varias limitaciones ahí el porque de mi proyecto.

La primera limitación es el mantenimiento, ya que cualquier alta, modificación o baja de una obra obliga a regenerar y volver a subir el PDF completo, lo que desincentiva mantener el catálogo actualizado. La segunda es la accesibilidad: los documentos PDF son poco compatibles con tecnologías de asistencia como lectores de pantalla, modos de alto contraste o navegación por teclado. La tercera es la calidad visual: las imágenes embebidas se reducen considerablemente para que el peso del PDF sea razonable, lo que impide apreciar el detalle real de las obras. Y la cuarta es la discontinuidad estética: el formato PDF rompe la experiencia de navegación al separarse del diseño de la web del instituto.

El público objetivo de la solución se divide en dos perfiles bien diferenciados. Por una parte, los visitantes externos (alumnado, familias, investigadores e interesados en el arte) que acceden de forma anónima para consultar la colección. Por otra, el personal del instituto responsable de mantener el catálogo, que necesita poder añadir, modificar y eliminar obras de forma cómoda y sin intervención técnica externa.

4.2. Estado del arte

A nivel internacional existen plataformas consolidadas para la difusión digital de colecciones artísticas, como Google Arts & Culture o los portales propios de grandes museos (Museo del Prado, Reina Sofía, Museo Británico, Louvre, etc.). Estas plataformas ofrecen funcionalidades muy avanzadas: búsquedas semánticas, recorridos virtuales en 360°, integración con sistemas de catalogación museística profesional como Domus o The Museum System, y aplicaciones móviles dedicadas.

En el ámbito específico de los centros educativos, la difusión de las colecciones artísticas suele resolverse con soluciones mucho más sencillas: páginas estáticas integradas dentro del propio sitio web del centro, documentos PDF descargables o, en algunos casos, galerías genéricas implementadas con plugins de WordPress u otras plataformas similares. No es habitual encontrar aplicaciones web a medida diseñadas específicamente para gestionar y exponer el patrimonio artístico de un IES.

Esta es precisamente la situación de partida del I.E.S Bernaldo de Quirós, cuyo catálogo se distribuye únicamente como PDF según se describe en el apartado anterior. El proyecto propuesto no pretende competir con las plataformas internacionales mencionadas, sino ofrecer una solución a medida que mejore el sistema actual del centro: una aplicación web sencilla, mantenible por el propio personal del instituto y diseñada específicamente para sus necesidades, que sustituye el PDF estático por un catálogo digital interactivo, accesible y actualizable en tiempo real.

5. Planificación

5.1. Recursos humanos y materiales

Recursos Humanos

El equipo de recursos humanos está compuesto por una única persona realizando todas las tareas de programación y mantenimiento de la aplicación web. Roles asumidos por Pelayo Prieto Calderón:

- **Desarrollador Backend**

- Diseño e implementación de la API REST
- Configuración de Spring Security y cadena de filtros JWT
- Diseño e implementación de la seguridad con JWT
- Creación de tests mediante JUnit

- **Desarrollador Frontend**

- Creación de la interfaz pública de la galería
- Desarrollo del panel de administración
- Diseño responsive de la interfaz con Bootstrap 5
- Implementación de la autenticación en el cliente
- Gestión de imágenes en el formulario de obra

- **Desarrollador/Administrador de Base de Datos**

- Diseño del modelo relacional y diagrama E-R (entidades del catálogo y del sistema de autenticación)
- **Analista**
 - Identificación de requisitos funcionales y no funcionales
 - Análisis de accesibilidad y usabilidad
 - Identificación y diseño de los casos de uso
 - Elaboración de diagramas técnicos (casos de uso, clases, componentes y despliegue)

Materiales

Hardware

El desarrollo del proyecto se ha llevado a cabo en un único equipo personal con sistema operativo Windows. Este equipo ha sido utilizado para todas las tareas: programación, pruebas, gestión de la base de datos y documentación.

Software

Las herramientas de software utilizadas se agrupan por categoría:

- **Entorno de desarrollo:** IntelliJ IDEA (backend) y Visual Studio Code (frontend) — ambos gratuitos en su versión Community/estándar.
- **Tecnologías del proyecto:** Java con Spring Boot (backend), Angular 21 (frontend), MySQL (base de datos) — todas gratuitas y de código abierto.
- **Servidor de base de datos:** MySQL Server y MySQL Workbench para la gestión visual — gratuitos.
- **Control de versiones:** Git y GitHub — gratuitos.
- **Pruebas de la API:** Postman — gratuito en su versión básica.
- **Documentación y diagramas:** Microsoft Word (documentación), plantuml.com, Claude.ai, MySQL Workbench *reverse engineer*

5.2. Presupuesto

El grosor del presupuesto de este proyecto sería el sueldo del programador, y el mantenimiento del código y alojamiento de la aplicación web.

Concepto	Coste
Desarrollo (3 meses, 1 programador al SMI)	3.663,00 €
Amortización del equipo informático (3 meses)	62,50 €
Alojamiento en servidor (1 año)	120,00 €
Software y herramientas de desarrollo	0,00 €
TOTAL	3.845,50 €

6. Análisis del sistema

6.1. Definición de requisitos

Requisitos Funcionales

Subsistema de autenticación y autorización

Código	Tipo	Descripción
RF1	Funcionalidad	El administrador puede iniciar sesión introduciendo su email y contraseña, y recibe un token de acceso JWT y un refresh token para acceder a los recursos protegidos.
RF2	Funcionalidad	La sesión se puede renovar automáticamente mediante el refresh token, sin pedir las credenciales otra vez al administrador.
RF3	Funcionalidad	El administrador puede cerrar sesión, lo que invalida su refresh token en el servidor y borra los tokens del cliente.

Subsistema de gestión de obras (panel de administración)

Código	Tipo	Descripción
RF4	Funcionalidad	Se pueden listar todas las obras del catálogo en formato tabla, con paginación y un buscador que filtra por título o por nombre del autor.
RF5	Funcionalidad	Se pueden crear nuevas obras introduciendo sus metadatos (título, autor, técnica, datación, dimensiones, descripción, etc.) y subiendo una o varias imágenes asociadas.

Código	Tipo	Descripción
RF6	Funcionalidad	Se pueden editar los datos de una obra existente, incluyendo cambiar de autor, de técnica, modificar metadatos y gestionar sus imágenes.
RF7	Funcionalidad	Se pueden eliminar obras del catálogo mediante borrado lógico, de manera que el registro deja de aparecer pero permanece en la base de datos.
RF8	Funcionalidad	Se pueden asociar una o varias imágenes a una obra subiéndolas desde el formulario, quedando almacenadas en el servidor y vinculadas a la obra.
RF9	Funcionalidad	Se pueden eliminar imágenes asociadas a una obra, borrando tanto el registro de la base de datos como el fichero del servidor.

Subsistema de gestión de autores (panel de administración)

Código	Tipo	Descripción
RF10	Funcionalidad	Se pueden listar los autores registrados, con un buscador que filtra por nombre o apellidos.
RF11	Funcionalidad	Se pueden crear nuevos autores registrando su nombre, apellidos, año de nacimiento, año de muerte, corriente artística y lugar de nacimiento.
RF12	Funcionalidad	Se pueden editar los datos de un autor existente.
RF13	Funcionalidad	Se pueden eliminar autores mediante borrado lógico, conservando las obras asociadas para preservar el histórico.

Subsistema de gestión de técnicas (panel de administración)

Código	Tipo	Descripción
RF14	Funcionalidad	Se pueden listar las técnicas registradas.
RF15	Funcionalidad	Se pueden crear nuevas técnicas para asociar luego a las obras del catálogo.
RF16	Funcionalidad	Se puede editar el nombre de una técnica existente.
RF17	Funcionalidad	Se pueden eliminar técnicas mediante borrado lógico, conservando las obras existentes que las usan.

Subsistema de consulta pública (visitante)

Código	Tipo	Descripción
RF18	Funcionalidad	Cualquier visitante puede consultar el catálogo público de obras en formato galería, sin necesidad de autenticarse.
RF19	Funcionalidad	El visitante puede buscar obras introduciendo el título o parte de él.
RF20	Funcionalidad	El visitante puede filtrar las obras del catálogo por autor.
RF22	Funcionalidad	El visitante puede abrir la ficha detallada de cualquier obra y ver sus imágenes en mayor resolución mediante una galería navegable.

Requisitos no funcionales

Se han categorizado los requisitos no funcionales según el atributo de calidad al que dan respuesta (rendimiento, usabilidad, accesibilidad, seguridad, compatibilidad, mantenibilidad, internacionalización y fiabilidad).

Código	Tipo	Descripción
RNF1	Rendimiento	Las consultas del catálogo respondes en menos de 1 segundo con el volumen actual de 258 obras y en condiciones normales de carga.
RNF2	Usabilidad	La interfaz es responsive y se adapta correctamente a pantallas de entre 320 píxeles (móvil) y 1920 píxeles (escritorio).
RNF3	Accesibilidad	La zona pública del sitio ofrece un nivel de accesibilidad adecuado: contraste suficiente, textos alternativos en todas las imágenes, jerarquía semántica de encabezados y navegación por teclado.
RNF4	Seguridad	Las contraseñas de los administradores se almacenan cifradas con BCrypt, nunca en texto plano.
RNF5	Seguridad	Los tokens de acceso JWT tienen una vigencia máxima de 15 minutos y la sesión se renueva mediante un refresh token de 7 días almacenado en la base de datos.
RNF6	Seguridad	El servidor solo acepta peticiones desde los orígenes autorizados (política CORS restrictiva).

Código	Tipo	Descripción
RNF7	Seguridad	El servidor valida que las imágenes subidas sean JPG, PNG o WEBP y no superen los 20 MB por fichero; los archivos no válidos se rechazan.
RNF8	Mantenibilidad	Las eliminaciones de obras, autores y técnicas se aplican como borrado lógico mediante el campo estado_borrado, para preservar el histórico.
RNF9	Internacionalización	La interfaz, los mensajes de error y los textos visibles al usuario están en castellano.
RNF10	Compatibilidad	La aplicación funciona correctamente en las versiones actualizadas de Chrome, Firefox, Edge y Safari, tanto en escritorio como en móvil.
RNF11	Fiabilidad	El backend gestiona los errores de manera centralizada y devuelve respuestas HTTP con códigos semánticamente correctos (400, 401, 403, 404, 500) y un mensaje descriptivo, que el frontend muestra al usuario de manera comprensible.
RNF12	Mantenibilidad	El sistema sigue una arquitectura cliente-servidor desacoplada, con separación entre el frontend (Angular) y el backend (Spring Boot expuesto como API REST), de modo que ambos se pueden desplegar y evolucionar de forma independiente.

6.2. Identificación de los subsistemas

Subsistema de autenticación y autorización

Gestiona el acceso al panel de administración. Es transversal al resto: cualquier operación de gestión del catálogo o de las imágenes pasa primero por este subsistema, que decide si la petición puede o no continuar.

Responsabilidad	Verificar la identidad del administrador, emitir y validar tokens JWT, renovar la sesión sin pedir credenciales repetidamente y proteger los endpoints según los permisos del usuario autenticado.
Componentes principales	En el backend: las entidades User, Role y RefreshToken; el AuthController que expone los endpoints de login, refresh y logout; el AuthService que orquesta la autenticación; el JwtService que firma y verifica los tokens; el JwtAuthFilter que intercepta cada petición protegida; el UserDetailsServiceImpl que carga los usuarios desde la base de datos; y el SecurityConfig que define la cadena de filtros y las reglas de autorización. En el frontend: el componente Login, el servicio Auth que conserva el token en sessionStorage, el AuthInterceptor que añade la cabecera Authorization a cada petición HTTP y el AuthGuard que protege las rutas del panel.
Tecnologías clave	Spring Security 6, JWT (firma de tokens), BCrypt (cifrado de contraseñas), sessionStorage del navegador.
Requisitos asociados	RF1, RF2, RF3, RNF4, RNF5, RNF6.

Subsistema de gestión del catálogo

Concentra las operaciones de administración sobre las tres entidades principales del catálogo: obras, autores y técnicas. Los tres siguen el mismo patrón arquitectónico (entidad → repositorio → servicio → controlador, expuesto en la API REST), de manera que se documentan juntos.

Responsabilidad	Permitir al administrador dar de alta, modificar, listar y eliminar (mediante borrado lógico) las obras, los autores y las técnicas que componen el catálogo del museo.
Componentes principales	En el backend: las entidades Obra, Autor y Tecnica con sus respectivos repositorios JPA, servicios (ObraService, AutorService, TecnicaService) que aplican la lógica de negocio y construyen los DTO de salida, y controladores (ObraController, AutorController, TecnicaController) que exponen los endpoints REST. En el frontend: las pantallas del panel admin con sus listados, formularios y servicios HTTP correspondientes (Obras, Autores, Tecnicas).
Tecnologías clave	Spring Data JPA, Hibernate 6 (incluida la anotación @SQLRestriction para el borrado lógico), Angular Reactive Forms, signals de Angular 21 para el estado local.
Requisitos asociados	RF4, RF5, RF6, RF7 (gestión de obras); RF10, RF11, RF12, RF13 (gestión de autores); RF14, RF15, RF16, RF17 (gestión de técnicas); RNF8 (borrado lógico).

Subsistema de gestión de imágenes

Se ocupa del tratamiento de archivos binarios (las imágenes de las obras). Está separado del subsistema de gestión del catálogo porque su modelo de almacenamiento es diferente: los metadatos viven en la base de datos relacional, pero el archivo en sí se guarda en el sistema de archivos del servidor.

Responsabilidad	Recibir, validar, almacenar y servir las imágenes asociadas a las obras del catálogo, así como permitir su eliminación tanto del sistema de archivos como del registro en base de datos.
Componentes principales	En el backend: la entidad Imagen, el ImagenController que expone los endpoints de subida y borrado, el ImagenService que

	coordina la persistencia con la base de datos, el FileStorageService que abstrae las operaciones sobre el sistema de archivos (generación de nombres únicos con UUID, validación de tipo MIME y tamaño) y el WebConfig que registra el resource handler que sirve las imágenes públicamente bajo la ruta /uploads/**. En el frontend: el formulario de obra incluye un selector múltiple de ficheros con vista previa antes del envío.
Tecnologías clave	Spring MultipartFile, sistema de archivos del servidor (carpeta uploads), HTML5 File API y URL.createObjectURL en el navegador.
Requisitos asociados	RF8, RF9, RNF7.

Subsistema de consulta pública

Es la cara visible del proyecto. Cualquier persona puede acceder sin autenticarse y navegar por la colección. Solo expone endpoints de lectura, lo que reduce de forma natural la superficie de ataque sobre los datos del catálogo.

Responsabilidad	Ofrecer al visitante anónimo una galería navegable del catálogo, con búsqueda por título, filtros por autor y por datación, y vista detallada de cada obra con su galería de imágenes en mayor resolución.
Componentes principales	En el backend: los endpoints GET de los controladores de Obra, Autor y Tecnica (marcados como permitAll en SecurityConfig). En el frontend: las páginas públicas con el listado/galería, el componente de búsqueda y filtros y la ficha individual de obra con visor de imágenes.

Tecnologías clave	Spring Boot (endpoints REST públicos), Angular 21 con componentes standalone y signals, Bootstrap 5 para el diseño responsive.
Requisitos asociados	RF18, RF19, RF20, RF21, RF22, RNF2, RNF3, RNF10.

Subsistema de autenticación y autorización

Gestiona el acceso al panel de administración. Es transversal al resto: cualquier operación de gestión del catálogo o de las imágenes pasa primero por este subsistema, que decide si la petición puede o no continuar.

Responsabilidad	Verificar la identidad del administrador, emitir y validar tokens JWT, renovar la sesión sin pedir credenciales repetidamente y proteger los endpoints según los permisos del usuario autenticado.
Componentes principales	En el backend: las entidades User, Role y RefreshToken; el AuthController que expone los endpoints de login, refresh y logout; el AuthService que orquesta la autenticación; el JwtService que firma y verifica los tokens; el JwtAuthFilter que intercepta cada petición protegida; el UserDetailsServiceImpl que carga los usuarios desde la base de datos; y el SecurityConfig que define la cadena de filtros y las reglas de autorización. En el frontend: el componente Login, el servicio Auth que conserva el token en sessionStorage, el AuthInterceptor que añade la cabecera Authorization a cada petición HTTP y el AuthGuard que protege las rutas del panel.
Tecnologías clave	Spring Security 6, JWT (firma de tokens), BCrypt (cifrado de contraseñas), sessionStorage del navegador.
Requisitos asociados	RF1, RF2, RF3, RNF4, RNF5, RNF6.

Subsistema de gestión del catálogo

Concentra las operaciones de administración sobre las tres entidades principales del catálogo: obras, autores y técnicas. Los tres siguen el mismo patrón arquitectónico (entidad - repositorio - servicio - controlador expuesto mediante API REST).

Responsabilidad	Permitir al administrador dar de alta, modificar, listar y eliminar (mediante borrado lógico) las obras, los autores y las técnicas que componen el catálogo del museo.
Componentes principales	En el backend: las entidades Obra, Autor y Tecnica con sus respectivos repositorios JPA, servicios (ObraService, AutorService, TecnicaService) que aplican la lógica de negocio y construyen los DTO de salida, y controladores (ObraController, AutorController, TecnicaController) que exponen los endpoints REST. En el frontend: las pantallas del panel admin con sus listados, formularios y servicios HTTP correspondientes (Obras, Autores, Tecnicas).
Tecnologías clave	Spring Data JPA, Hibernate 6 (incluida la anotación @SQLRestriction para el borrado lógico), Angular Reactive Forms, signals de Angular 21 para el estado local.
Requisitos asociados	RF4, RF5, RF6, RF7 (gestión de obras); RF10, RF11, RF12, RF13 (gestión de autores); RF14, RF15, RF16, RF17 (gestión de técnicas); RNF8 (borrado lógico).

Subsistema de gestión de imágenes

Se ocupa del tratamiento de archivos binarios (las imágenes de las obras). Está separado del subsistema de gestión del catálogo porque su modelo de almacenamiento es diferente: los metadatos viven en la base de datos relacional, pero el archivo en sí se guarda en el sistema de archivos del servidor.

Responsabilidad	Recibir, validar, almacenar y servir las imágenes asociadas a las obras del catálogo, así como permitir su eliminación tanto del sistema de archivos como del registro en base de datos.
Componentes principales	En el backend: la entidad Imagen, el ImagenController que expone los endpoints de subida y borrado, el ImagenService que coordina la persistencia con la base de datos, el FileStorageService que abstrae las operaciones sobre el sistema de archivos (generación de nombres únicos con UUID, validación de tipo MIME y tamaño) y el WebConfig que registra el resource handler que sirve las imágenes públicamente bajo la ruta /uploads/**. En el frontend: el formulario de obra incluye un selector múltiple de ficheros con vista previa antes del envío.
Tecnologías clave	Spring MultipartFile, sistema de archivos del servidor (carpeta uploads), HTML5 File API y URL.createObjectURL en el navegador.
Requisitos asociados	RF8, RF9, RNF7.

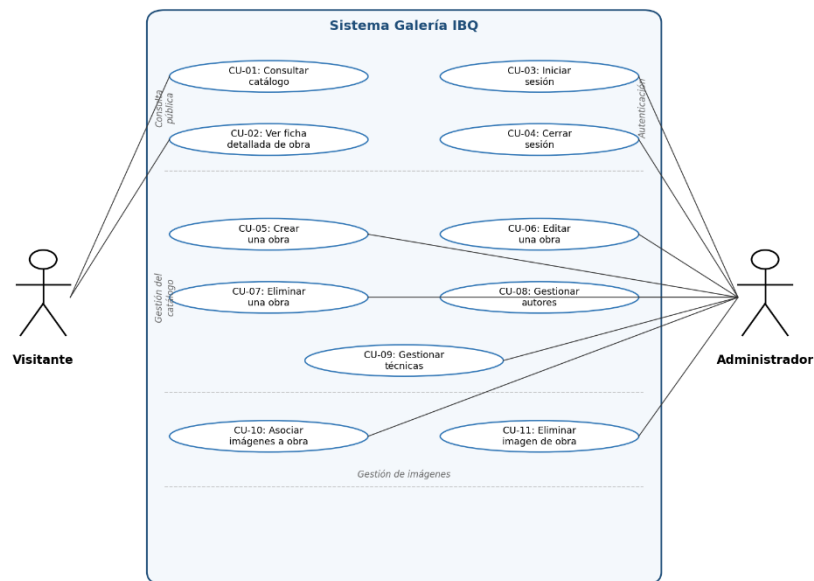
Subsistema de consulta pública

Es la cara visible del proyecto. Cualquier persona puede acceder sin autenticarse y navegar por la colección. Solo expone endpoints de lectura, lo que previene ataques sobre la colección de datos del catalogo.

Responsabilidad	Ofrecer al visitante una galería navegable del catálogo, con búsqueda por título, filtros por autor y vista detallada de cada obra con su galería de imágenes en mayor resolución.
Componentes principales	En el backend: los endpoints GET de los controladores de Obra, Autor y Tecnica (marcados como permitAll en SecurityConfig). En el frontend: las páginas públicas con el listado/galería, el

	componente de búsqueda y filtros y la ficha individual de obra con visor de imágenes.
Tecnologías clave	Spring Boot (endpoints REST públicos), Angular 21 con componentes standalone y signals, Bootstrap 5 para el diseño responsive.
Requisitos asociados	RF18, RF19, RF20, RF21, RF22, RNF2, RNF3, RNF10.

6.3. Análisis de casos de uso



6.4. Análisis de interfaces

Parte Administrador

Obras + Añadir Obra

VISTA	NOMBRE DE LA OBRA	AUTOR	FECHA DE CREACIÓN	ACCIONES
	La persistencia de la memoria	Salvador Dalí	1931	
	Noche estrellada	Vincent van Gogh	1889	
	Guernica	Pablo Picasso	1937	
	El grito	Edvard Munch	1893	
	Las meninas	Diego Velázquez	1656	

Mostrando 1-5 de 24 obras Anterior Siguiente

Panel de Administración - Gestión del Inventario de Obras © 2024

Listado de obras

Añadir Nuevo Objeto Cancelar Guardar Registro

Registro de catálogo para la base de datos de arte y artefactos

Subir fotografía principal

Ayuda de Registro

Asegúrese de que las dimensiones están expresadas en centímetros (alto x ancho x fondo). Use el campo de observaciones para detalles no contemplados en las categorías estándar.

IDENTIFICACIÓN BÁSICA

Título / Nombre del Objeto

Autor / Creador Datación

Tipología Dimensiones

Técnica y Materiales

Descripción

HISTORIA Y PROCEDENCIA

Fecha de Ingreso Modo de Ingreso

Procedencia

Marcas e Inscripciones

ESTADO Y UBICACIÓN

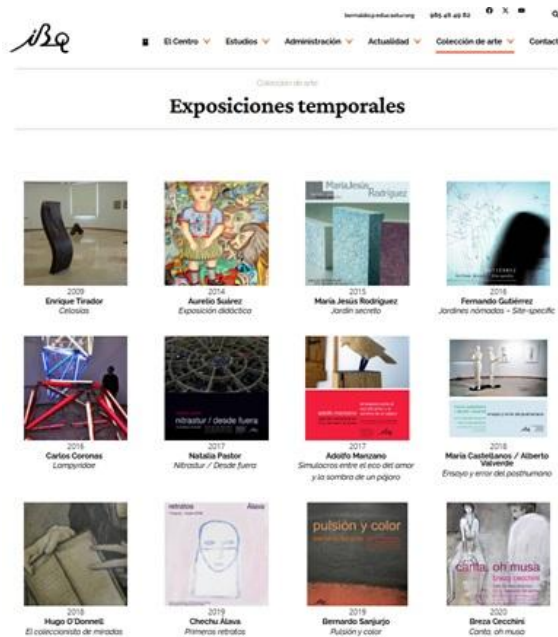
Estado de Conservación Ubicación Actual

Restauraciones

Observaciones

Descartar Cambios Finalizar Registro

Parte Pública



Inspiración en página de exposiciones temporales del IBQ

Formulario para añadir/editar obras

6.5. Especificación del plan de pruebas

En este apartado se define el plan de pruebas que se aplicará al sistema. La verificación del software no se entiende como una etapa aislada al final del desarrollo, sino como un conjunto de actividades complementarias que cubren las distintas dimensiones de la calidad: la corrección de la lógica interna, el comportamiento de la API expuesta, la integración completa frontend-backend, la accesibilidad de la zona pública, la usabilidad percibida por el usuario final y la compatibilidad con los navegadores y dispositivos más utilizados.

La estrategia combina pruebas automáticas (unitarias e integración de la API) con pruebas manuales (funcionales end-to-end, usabilidad y compatibilidad) y herramientas de auditoría automática (accesibilidad). Esta combinación se considera adecuada para un proyecto cliente-servidor de alcance acotado como el que se

documenta, donde el coste de una infraestructura completa de pruebas automatizadas E2E no se justifica frente al valor que aportan las pruebas manuales bien planificadas. Los resultados concretos de la ejecución se recogen en el apartado 9.

Pruebas unitarias del backend

Verifican el funcionamiento aislado de las clases del backend que contienen lógica de negocio, sin depender de la base de datos ni de servicios externos. El alcance se limita a los servicios del modelo de negocio principal: `ObraService`, `AutorService` y `TecnicaService`. Se cubren los métodos públicos de cada uno y los principales casos de error: excepciones lanzadas cuando un identificador no existe, validaciones, asignación automática del campo `estado_borrado` y mapeo entidad-DTO. Los servicios de soporte técnico (`JwtService` y `FileStorageService`) no se cubren en este apartado al estar mejor verificados desde fuera: el primero mediante las pruebas de integración con Postman y el segundo mediante las pruebas funcionales end-to-end. Las pruebas se implementan con JUnit 6 y Mockito, y se ejecutan con el comando `mvn test`.

Pruebas de integración de la API REST

Verifican que los endpoints REST responden correctamente desde el punto de vista del cliente, ejecutando peticiones reales contra la aplicación en marcha y comprobando códigos HTTP, cuerpos de respuesta y persistencia de los cambios. El alcance incluye los endpoints críticos: autenticación (login, refresh, logout), CRUD completo de obras, autores y técnicas, y subida y borrado de imágenes. Se verifica además que los endpoints protegidos rechazan con 401 o 403 las peticiones sin token o con token expirado. Se utiliza Postman, con una colección guardada que reproduce la secuencia de peticiones representativas.

Pruebas funcionales end-to-end manuales

Verifican desde el punto de vista del usuario final que los casos de uso definidos en el apartado 6.3 se completan correctamente recorriendo el sistema desde la interfaz web, comprobando la integración entre frontend, backend, base de datos y sistema de archivos. Se recorren los once casos de uso paso a paso, incluyendo los flujos alternativos significativos (validación de campos obligatorios, manejo de errores del servidor, gestión de imágenes fuera de los formatos permitidos).

Pruebas de accesibilidad

Verifican que la zona pública del sitio ofrece un nivel de accesibilidad adecuado, según lo establecido en el requisito RNF3. Se realizan auditorías automáticas con la herramienta WAVE (WebAIM) sobre la portada con el catálogo y la ficha detallada de obra. Como criterio de éxito se establece una puntuación igual o superior a 8 sobre 10 en cada página pública.

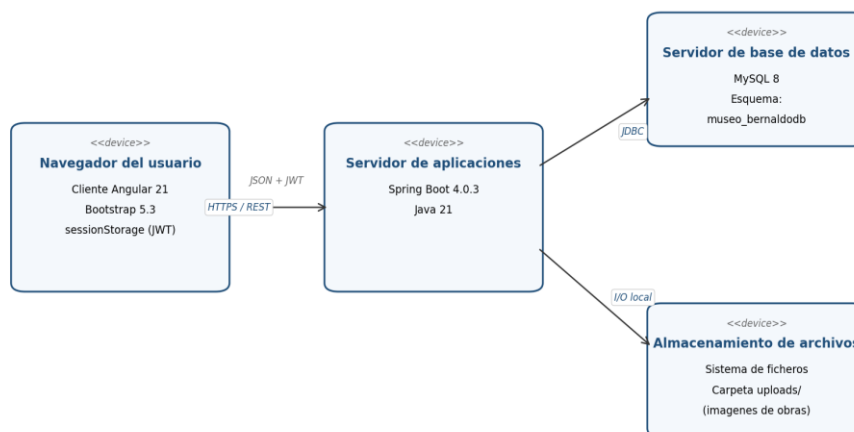
7. Diseño del sistema

7.1. Arquitectura del sistema

Diagrama de despliegue

El siguiente diagrama representa los cuatro nodos físicos que componen el sistema, junto con los protocolos que utilizan para comunicarse entre sí.

Diagrama de despliegue del sistema Galeria IBQ



El navegador del usuario aloja la aplicación cliente Angular y guarda en su sessionStorage los tokens JWT necesarios para autenticarse contra el servidor. El servidor de aplicaciones ejecuta el backend Spring Boot, que actúa como intermediario entre el cliente y los dos sistemas de almacenamiento. La base de datos MySQL persiste todas las entidades del catálogo (obras, autores, técnicas, etc.) además de los datos relativos a la autenticación de los administradores. El sistema de archivos

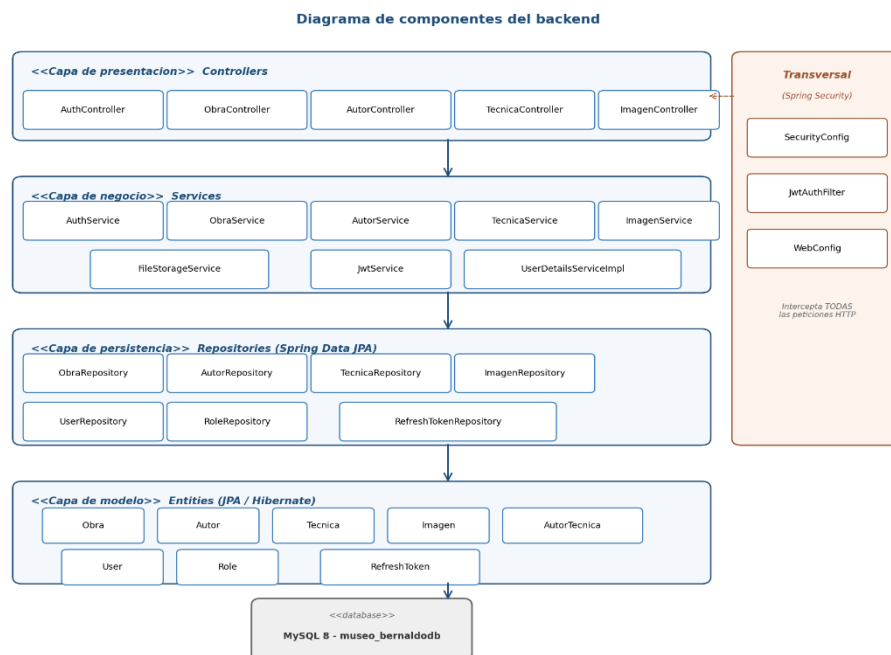
almacena las imágenes asociadas a las obras como ficheros independientes, organizados en una jerarquía de carpetas por identificador de obra.

La comunicación cliente-servidor se realiza mediante HTTP sobre el formato JSON, con tokens JWT en la cabecera Authorization para las peticiones autenticadas. El servidor se comunica con la base de datos mediante JDBC a través del controlador oficial de MySQL, y con el sistema de archivos mediante operaciones de entrada y salida estándar de la plataforma Java.

Es importante señalar que, aunque durante el desarrollo los cuatro nodos conviven en la misma máquina física, la separación lógica permite que en un despliegue de producción puedan distribuirse en máquinas distintas sin modificar la arquitectura: el servidor de aplicaciones podría escalar horizontalmente, la base de datos podría ejecutarse en un servidor dedicado y el almacenamiento de imágenes podría externalizarse a un servicio especializado.

Diagrama de componentes del backend

El backend está organizado siguiendo un patrón de arquitectura en capas. Cada capa tiene una responsabilidad bien delimitada y solo se comunica con la capa inmediatamente inferior, lo que facilita el mantenimiento, las pruebas y la evolución del código.



La capa de presentación está formada por los controladores REST (Controllers), que son los puntos de entrada al sistema desde el exterior. Cada controlador expone los endpoints HTTP de un recurso concreto (obras, autores, técnicas, imágenes y autenticación) y delega la lógica de negocio en los servicios correspondientes. Su responsabilidad se limita a la traducción entre el formato HTTP y las llamadas internas.

La capa de negocio contiene los servicios (Services), donde reside la lógica funcional de la aplicación: validaciones, mapeos entre entidades y DTOs, asignación del campo de borrado lógico, gestión de tokens JWT y operaciones sobre el sistema de archivos para las imágenes. Los servicios de soporte (FileStorageService, JwtService, UserDetailsServiceImpl) se separan visualmente porque, a diferencia de los servicios de modelo, no representan una entidad del catálogo sino una funcionalidad técnica transversal.

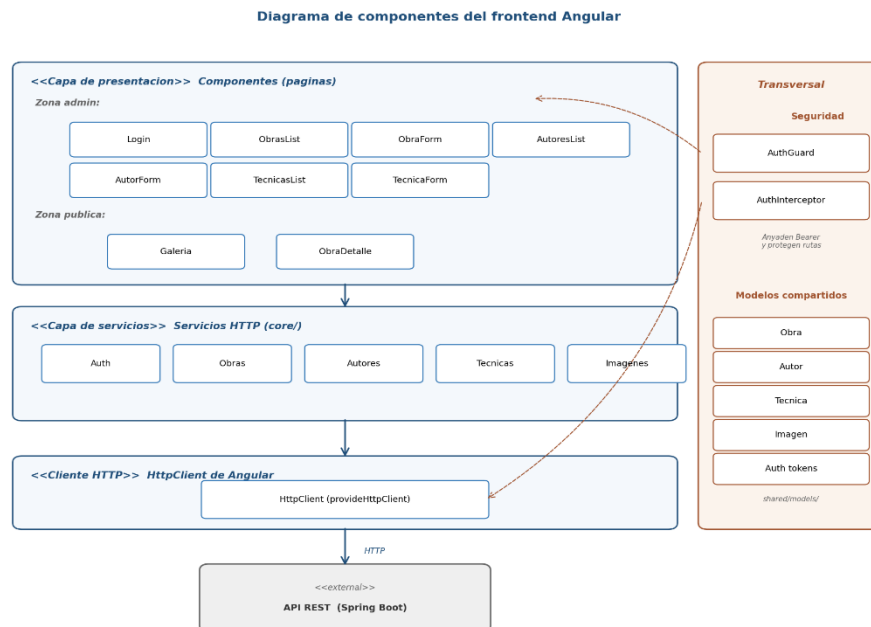
La capa de persistencia agrupa los repositorios de Spring Data JPA. Cada repositorio extiende de la interfaz JpaRepository y Spring genera automáticamente su implementación en tiempo de arranque, evitando tener que escribir consultas SQL manuales para las operaciones CRUD básicas.

La capa de modelo contiene las entidades JPA, que representan las tablas de la base de datos como clases Java anotadas con @Entity. Estas entidades son las que Hibernate persiste a través de los repositorios.

A la derecha del diagrama aparece una franja transversal con los componentes de Spring Security: SecurityConfig (que define las reglas de autorización), JwtAuthFilter (que intercepta cada petición HTTP entrante para validar el token JWT) y WebConfig (que registra el resource handler para servir las imágenes públicamente). Estos componentes se representan en una franja separada porque no encajan en ninguna capa concreta: actúan antes de que la petición llegue al controlador, en una posición transversal al flujo principal.

Diagrama de componentes del frontend

El frontend Angular sigue una organización en capas similar a la del backend, aunque con tres niveles en lugar de cuatro: los componentes de presentación, los servicios HTTP que encapsulan las llamadas a la API y el cliente HTTP nativo de Angular que realiza las peticiones reales.



La capa de presentación está formada por nueve componentes Angular standalone, agrupados visualmente en dos zonas: la zona de administración (con los componentes de login y los listados y formularios de obras, autores y técnicas) y la zona pública (con la galería y la ficha detallada de obra). Cada componente se carga de forma perezosa mediante loadComponent en su ruta correspondiente, lo que mantiene el bundle inicial pequeño y agiliza el primer renderizado.

La capa de servicios HTTP, ubicada en la carpeta core, agrupa los cinco servicios inyectables que encapsulan las llamadas a la API. Cada componente que necesita datos del backend inyecta el servicio correspondiente; los componentes no llaman directamente a HttpClient, sino que esa responsabilidad recae siempre en los servicios. Esto sigue el mismo principio de separación de responsabilidades que aplica el

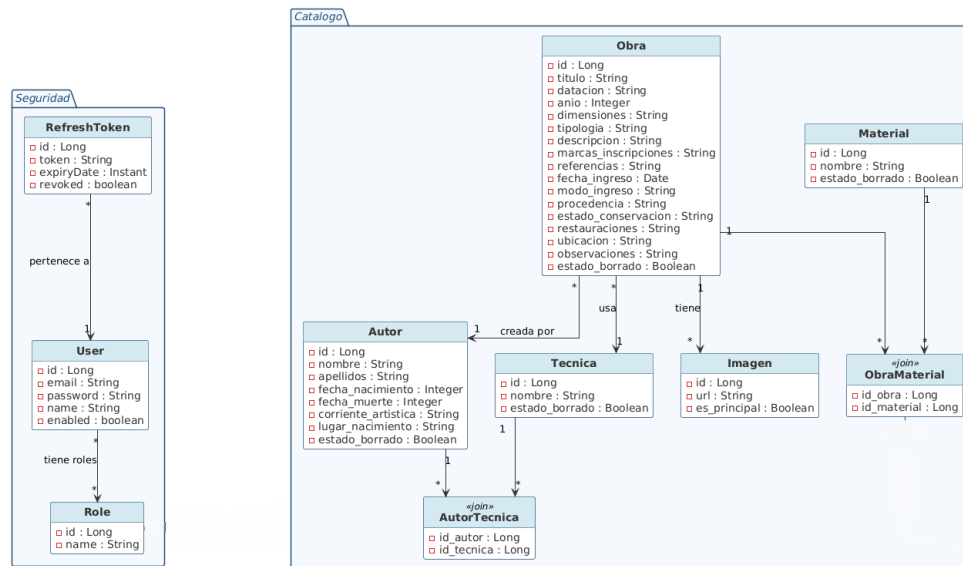
backend: los componentes se centran en la presentación, los servicios en la comunicación con el servidor.

Por debajo, el `HttpClient` nativo de Angular es el que realiza las peticiones HTTP reales. Está configurado en `app.config.ts` mediante `provideHttpClient`, donde se registra el interceptor de autenticación que se describe a continuación.

La franja transversal a la derecha agrupa dos bloques. El primero, denominado Seguridad, contiene `AuthGuard` (que protege las rutas del panel administrativo redirigiendo a la pantalla de login cuando no hay sesión activa) y `AuthInterceptor` (que añade automáticamente la cabecera `Authorization` con el token JWT a todas las peticiones HTTP salientes). Las flechas discontinuas indican que estos componentes actúan en los márgenes del flujo principal: el interceptor sobre el `HttpClient`, el guard sobre las rutas hacia los componentes. El segundo bloque, Modelos compartidos, agrupa las interfaces TypeScript de la carpeta `shared/models`, que tipan los datos que viajan entre cliente y servidor.

La correspondencia entre las capas del frontend y las del backend es deliberada: cada servicio HTTP del cliente se comunica con su controlador equivalente del servidor, y cada interfaz de modelos coincide con la estructura de los DTOs definidos en el backend. Esta simetría facilita el mantenimiento al tener un contrato claro y consistente entre ambos extremos.

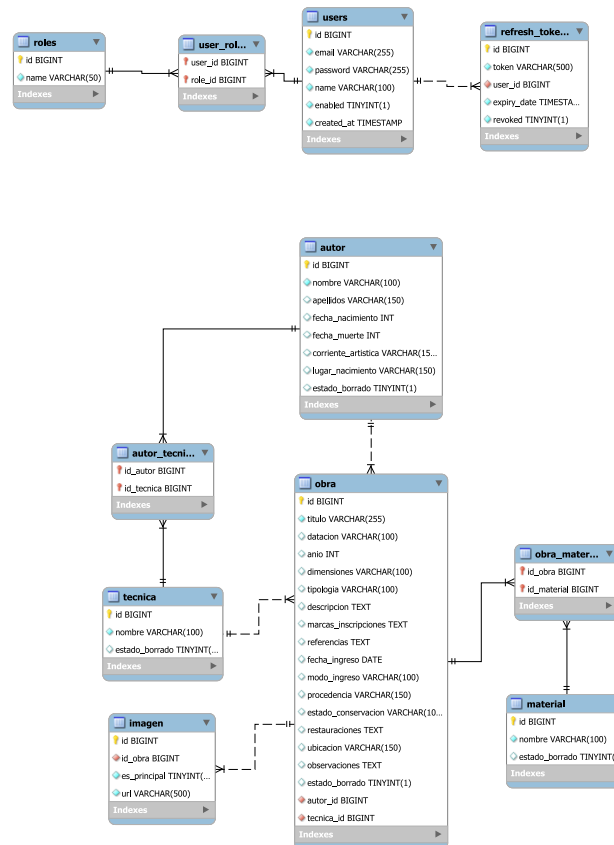
7.2. Diseño de clases



La estructura de clases de cualquier galería contiene en su centro las obras en si. En este caso, los datos necesarios para almacenar las obras vienen dados como requisitos por el usuario. Todas las demás entidades son están pensadas para facilitar la vida al usuario, tanto administrador como usuario estándar, a la hora de filtrar las obras.

Como se puede observar, las tablas relacionadas con la seguridad (Spring Security) están totalmente separadas en la base de datos del resto de entidades.

7.3. Diseño de la base de datos



7.4. Diseño de interfaces

El diseño de las interfaces de la parte pública proviene de una sección de exposiciones que ya contienen la página del Instituto Bernaldo de Quirós. El resto de interfaces, provienen de creación propia.

Parte pública:



El Centro

Estudios

Administración

Actualidad

Colección de arte

Contacto



2009
Enrique Tirador
Celosías



2014
Aurelio Suárez
Exposición didáctica



2015
María Jesús Rodríguez
Jardín secreto



2016
Fernando Gutiérrez
Jardines nómadas - Site-specific



2016
Carlos Coronas
Lampridae



2017
Natalia Pastor
Nitrastur / Desde fuera



2017
Adolfo Manzano
Simulacros entre el eco del amor y la sombra de un pájaro



2018
María Castellanos / Alberto Valverde
Ensayo y error del posthumano



El Centro

Estudios

Administración

Actualidad

Colección de arte

Contacto

bernaldo@educastur.org

985 46 49 82



Museo de arte

Celosías - Enrique Tirador

[← Volver al índice de exposiciones](#)



Canta, oh musa

[Descargar](#)

Díptico de la exposición

[Descargar](#)

Enrique Tirador
IBQ
viernes 29 de mayo de 2009

Enrique Tirador es escultor sólido que ha expuesto en importantes salas de nuestra región y del territorio nacional, destacan sus piezas de escultura urbana colocadas en plazas e instituciones. Son interesantes sus colaboraciones con otros artistas como la que le permitió trabajar con Agustín Ibarrola en el Proyecto de los Olmos en Salamanca.

Complementa su trabajo escultórico con pintura y obra gráfica. En 1996 obtuvo la Medalla de Oro y Primer Premio en el XXVII Certamen Nacional de Pintura de Lueca y en 2004 la Beca Antón-Ayuda a la creación escultórica. Se pueden contemplar algunas obras suyas en el Ayuntamiento de Carreño, en el Paseo Marítimo de San Antonio, Candás, y en el Museo Pérez Comendador de Extremadura. Además de su intensa carrera artística, Tirador dedica profunda vocación a la docencia, impartiendo diversos talleres y dando clases en Centros de Educación Secundaria de Asturias. Actualmente está montando un nuevo taller en Piedras Blancas que parece ser su espacio definitivo de inspiración.

Para la exposición del IES Bernaldo de Quirós, muestra bajo el título Celosías un estudiado conjunto escultórico formado por ocho piezas en hierro distribuidas en dos espacios. Con ella plantea el hecho íntimo de ver sin ser visto y la necesidad de ocultar la verdadera identidad para protegerse. La Sala de Exposiciones, conocida como la Rotonda por su planta circular, abraza, en sus propias palabras, lo que recuerda un selecto jardín de esculturas, por sus formas ondulantes, que parecen emergen de lo más profundo de la tierra. En el patio del edificio histórico expone otro grupo de piezas de menor escala. En él, hostiles volúmenes desafían a la gravedad con formas curvas que ayudan a hacerlas más ligeras. A semejanza corazas y yelmos con celosías por las que respirar, pues las aperturas geométricas y vegetales tienden a abrirse hueco, buscando la libertad en la prisión de los pesados sólidos. La intervención de Enrique Tirador en nuestro centro se complementó con una charla dirigida a alumnos del centro y público en general, en la que el artista explicó cómo es su proceso de creación, centrándose en el concepto de espacio y el complejo proceso de planificación que requiere desarrollar un encargo de escultura urbana.

Parte Administrador:

Obras

Buscar por nombre o autor...

Añadir Obra

VISTA	NOMBRE DE LA OBRA	AUTOR	FECHA DE CREACIÓN	ACCIONES
	La persistencia de la memoria	Salvador Dalí	1931	 
	Noche estrellada	Vincent van Gogh	1889	 
	Guernica	Pablo Picasso	1937	 
	El grito	Edvard Munch	1893	 
	Las meninas	Diego Velázquez	1656	 

Mostrando 1-5 de 24 obras

Anterior

Siguiente

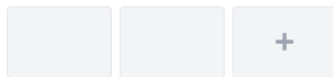
Panel de Administración - Gestión de Inventario de Obras © 2024

Añadir Nuevo Objeto

Registro de catálogo para la base de datos de arte y artefactos

Cancelar

Guardar Registro



i Ayuda de Registro

Asegúrese de que las dimensiones estén expresadas en centímetros (alto x ancho x fondo). Use el campo de observaciones para detalles no contemplados en las categorías estándar.

IDENTIFICACIÓN BÁSICA

Título / Nombre del Objeto

Ej. Retrato de caballero

Autor / Creador

Datación

Ej. Siglo XVII (c. 1640)

Tipología

Ej. Pintura al óleo

Dimensiones

Ej. 120 x 85 cm

Técnica y Materiales

Ej. Óleo sobre lienzo

Descripción

HISTORIA Y PROCEDENCIA

Fecha de Ingreso

Modo de Ingreso

Procedencia

Origen anterior del objeto

Marcas e Inscripciones

Firmas, sellos, etiquetas posteriores...

ESTADO Y UBICACIÓN

Estado de Conservación

Ubicación Actual

Sala, vitrina, almacén...

Restauraciones

Historial de intervenciones

Observaciones

Descartar Cambios

Finalizar Registro

8. Implementación

8.1. Lenguajes de programación

Backend:

- Java 21.0.9
- Spring Boot 4.0.3
- Spring Security
- MySQL 8.0.44

Frontend

- Node.js 24.15.0
- Package Manager (npm) 11.12.1
- Angular 21.2.13
- Bootstrap 5.3.8

8.2. Herramientas utilizadas

- Visual Studio Code
- IntelliJ IDEA Community Edition
- MySQL Workbench
- Claude.ai

9. Desarrollo de las pruebas

9.1. Pruebas de código

Para la verificación automatizada de la lógica de negocio se han implementado pruebas unitarias con JUnit 6 (Jupiter) y Mockito siguiendo el patrón AAA (Arrange-Act-Assert). Las pruebas se ejecutan con el comando `mvn test` integrado en el ciclo de vida de Maven.

Se han implementado veintiocho pruebas unitarias distribuidas en tres clases de test, una por cada servicio del modelo de negocio principal.

TecnicaServiceTest (8 pruebas)

Cubre todas las operaciones del servicio: alta con asignación automática del campo `estado_borrado` a `false`, listado completo, listado con base de datos vacía como caso límite, búsqueda por identificador existente y no existente (lanzando la excepción correspondiente), actualización y eliminación lógica. Se verifica también, mediante `verify(...).never()`, que cuando la técnica buscada no existe el método `save` no llega a invocarse, garantizando la integridad.

AutorServiceTest (8 pruebas)

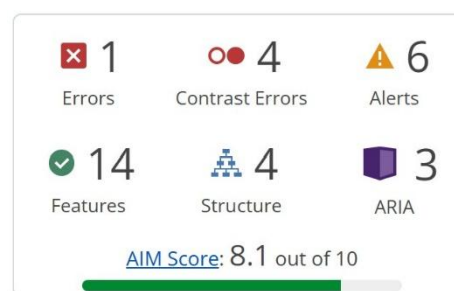
Sigue el mismo patrón que TecnicaServiceTest con la particularidad de verificar el mapeo correcto de los siete campos del AutorDTO (nombre, apellidos, fecha_nacimiento, fecha_muerte, corriente_artistica, lugar_nacimiento e identificador). Se introduce un método auxiliar privado crearAutorEjemplo(Long id) para evitar repetir los siete setters en cada test.

ObraServiceTest (12 pruebas)

Es la clase de test más compleja al cubrir el servicio que más dependencias tiene (cruza tres repositorios: obras, autores y técnicas). Se prueban tanto el flujo correcto (alta con identificadores válidos) como los flujos de error (autorId inexistente, tecnicaId inexistente, obra inexistente al actualizar o eliminar). Incluye un test específico que verifica el mapeo de las imágenes asociadas a una obra, comprobando que el método de mapeo gestiona correctamente tanto el caso con imágenes como sin ellas. Se utiliza thenAnswer en lugar de thenReturn para simular el comportamiento de JPA al guardar una entidad.

9.2. Pruebas de accesibilidad

La auditoría de accesibilidad de la zona pública se ha realizado con la herramienta WAVE (WebAIM), evaluando la portada con el catálogo y la ficha detallada de obra. La puntuación obtenida es de 8,1 sobre 10, por encima del umbral mínimo de 8 fijado como criterio de éxito.



10. Conclusiones

10.1. Desarrollo del proyecto

El desarrollo no presentó ningún obstáculo insalvable, pero sí dos focos de dificultad que concentraron buena parte del esfuerzo y del tiempo invertido: la implementación de la seguridad en el backend y el aprendizaje del framework de frontend.

Implementación de la seguridad con Spring Security

La parte que más tiempo requirió fue la configuración de la seguridad con Spring Security. Al optar por un esquema de autenticación sin estado basado en JSON Web Tokens (JWT), en lugar del login por sesión que Spring ofrece por defecto, fue necesario construir manualmente varias piezas y entender cómo encajan entre sí: un filtro propio (JwtAuthFilter) que intercepta cada petición y se ejecuta antes del filtro estándar de autenticación, un servicio (JwtService) para generar y validar los tokens firmados, la integración con UserDetailsService y el AuthenticationManager, y la definición de la cadena de filtros y de las reglas de autorización (qué endpoints son públicos y cuáles exigen rol de administrador). A ello se sumaron detalles que al principio provocaban errores difíciles de diagnosticar, como la configuración de CORS para permitir las peticiones del cliente Angular o el orden de los filtros. La dificultad no estuvo tanto en escribir el código como en comprender el flujo completo de una petición autenticada; se resolvió apoyándose en la documentación oficial de Spring Security y construyendo la solución de forma incremental, probando cada pieza por separado.

Curva de aprendizaje de Angular

El segundo foco, más constante a lo largo de todo el proyecto, fue el aprendizaje de Angular: era el framework con el que menos experiencia previa se tenía y, además, se trabajó con una versión muy reciente (Angular 21) que introduce un modelo de desarrollo distinto al de versiones anteriores: componentes standalone, reactividad basada en signals y la nueva sintaxis de control de flujo en plantillas (@if, @for). Esto hizo que muchos de los ejemplos disponibles estuvieran desactualizados, obligando a recurrir continuamente a la documentación oficial, a tutoriales antiguos y al uso de la IA como fuentes. Los puntos de mayor fricción fueron la gestión del estado de la

galería con signals y propiedades computadas (filtros y paginación), los formularios reactivos del panel de administración —en especial el de obras, con la subida de varias imágenes y la distinción entre imágenes ya guardadas y pendientes— y la comunicación con el backend mediante el interceptor HTTP que añade el token. Se superó con práctica continua, descomponiendo cada pantalla en problemas pequeños hasta asimilar el flujo de trabajo del framework.

10.2. Futuras ampliaciones

Aunque el proyecto cumple los objetivos planteados, durante el desarrollo han surgido contratiempo o la ideación de áreas nuevas o que podrían recibir una mejora, las cuales se proponen como ampliaciones futuras.

Alta de autores, técnicas y materiales desde el formulario de obra

En la versión actual, para asignar a una obra un autor, una técnica o un material que todavía no existe en el sistema, el administrador debe abandonar el formulario de la obra, ir al apartado correspondiente, dar de alta la nueva entidad y volver a empezar la creación de la obra. Este flujo resulta poco cómodo, sobre todo al catalogar varias obras seguidas, y supone el riesgo de perder los datos ya introducidos. La ampliación propuesta consiste en permitir la creación de estas entidades sin salir del formulario de la obra: cuando el autor, la técnica o el material buscado no aparezca en el desplegable, una opción de "crear nuevo" abriría una ventana emergente (o un formulario incrustado) donde darlo de alta en el momento, de modo que quede disponible y seleccionado automáticamente al cerrarla, sin interrumpir la edición ni perder el progreso de la obra.

Filtrado avanzado del catálogo

La galería pública permite actualmente filtrar las obras por autor, técnica y material. Como ampliación se plantea enriquecer este sistema con criterios más granulares aprovechando información que ya se almacena en el modelo pero que todavía no se explota en la búsqueda, como la corriente artística o el lugar de nacimiento del autor, además de un filtrado por época o franja temporal a partir de la datación de las obras. Igualmente sería interesante poder combinar varios criterios

simultáneamente y reflejar los filtros activos en la URL para poder compartir o guardar una búsqueda concreta.

Servicio de Email para la recuperación de contraseñas

Actualmente si algún usuario se olvidase de su contraseña, haría falta acceder a la base de datos y cambiar manualmente su contraseña, encriptándola previamente mediante BCrypt. Esto puede ser tedioso, tanto para el técnico encargado de mantener la base de datos, como para el usuario que tiene que depender de la disponibilidad de dicho técnico para poder acceder el panel de administración.